

T10.3 - Big Data (recorrido inicial)

Recorrido inicial

1. **Estado inicial.** Una RDBMS simple con una tabla de pageviews (`user_id`, `url`, `date`, `pageviews`). Ante cada evento se actualiza el registro correspondiente.
2. **Aumenta el tráfico.** Si la DB *no puede mantener el ritmo de escrituras* → *timeouts*.
Solución: desacoplar componentes con una cola (queue) y un **worker** independiente que *acumula eventos para reducir llamadas* a la base. *Esto maneja la back-pressure*.
3. **Aumenta aún más el tráfico.** *El worker se atrasa y la cola solo crece*.
Solución: *paralelizar agregando más workers* con un **load balancer** que *distribuye la carga* equitativamente.
4. **La base de datos vuelve a ser cuello de botella.** Se aplica partición horizontal (**sharding**): *desparramar los datos en múltiples servidores de DB*. Cada servidor tiene un subconjunto de la tabla. La asignación se hace de una manera para que tenga *distribución uniforme*.
Las queries requieren consultar todos los shards y hacer merge de resultados. *Si la app crece más*, hay que hacer *resharding* sin dejar de recibir tráfico.
5. **Fallas en nodos.** *Más nodosencia de fallas de disco*. Se nec implica más frecuencia de fallas de disco.
Se necesita hacer la DB fault tolerant → *registrar* qué nodos están online *y* mantener réplicas pasivas*.
6. **Fallas en el código (bugs).** Un *bug en los workers puede corromper la información* (source of truth = DB). Los *backups no alcanzan* porque implican perder información y no se puede saber qué está corrupto.
Se necesita *human fault tolerance*.

First principles: el *objetivo* de un sistema de datos es *contestar preguntas basándose en información recolectada*. *No toda la información es igual*: cierta información es derivada de otra. Si se busca el origen, se llega a una fuente que no es derivable de ninguna otra.

Data → *información que es verdadera simplemente por existir*, la fuente desde la cual *toda otra información puede ser derivada*.

La definición más general de un sistema de datos es: `query = function(all data)`.

Entre los **requerimientos no funcionales** deseables están: (i) *tolerancia a fallos*, (ii) *escalabilidad*, (iii) *extensibilidad*, (iv) *mantenibilidad*, (v) *baja latencia*, y (vi) *soporte para consultas específicas*.

Soluciones de Data Storage

Data Warehouse y **Data Lake** son *arquitecturas con fines analíticos y de reporting*. Son *no transaccionales* (baja o nula mutabilidad), *repositorios centrales* que *unifican muchas fuentes diversas*, orientados a grandes volúmenes y pensados para escalabilidad.

Data Warehouse

Datos *fuertemente estructurados* con estrategia **schema-on-write**: las estructuras se definen *antes* de la carga y los datos se validan contra ellas.

La estructura se *orienta a los casos de análisis y reporting necesarios*, impuesta mediante ETL.

Ventajas de schema-on-write:

- Las *lecturas son más rápidas* (la penalización está en la carga).
- La *relación entre entidades es sencilla*.
- Los *datos quedan documentados en la estructura*.

Desventajas:

- Cualquier *cambio en el esquema obliga a reprocesar*.
- Es *complicado mapear datos no estructurados*.
- Es *costoso mantener el nivel de detalle más atómico para volúmenes altos*.

Incluye *normalización de datos*, *data cleaning* (imponer calidad, descartar lo que no encuadre) y *data governance/seguridad* robustas.

Componentes principales: *almacenamiento* (RDBMS en distintas capas), *pipelines de ETL*, *capa de metadata*, *data marts* (capa de consulta para casos específicos) y *capa de presentación* (análisis y reporting).

Data Lake

Datos crudos de las distintas fuentes con estrategia **schema-on-read**: no se impone estructura al ingestar (ELT), no se descartan ni limpian datos.

Acepta datos estructurados y no estructurados. Los datos son *inmutables*.

Ventajas de schema-on-read:

- Los *volcados son más rápidos*.
- Hay *flexibilidad para leer según la vista que interese* en cada consulta.
- Es *apropiado para datos no estructurados*.
- Permite *guardar el nivel de detalle más atómico*.

Desventajas:

- La *penalización está en la lectura*.

- *No hay documentación clara* de los datos.
- Necesita *alta capacidad de proceso*.

Tiene *más dificultades para lograr data governance y seguridad*. Sirve para *análisis y reporting* pero también para otros casos como *Machine Learning*.

Pipelines de ingestión

ETL (Extract, Transform, Load)

Proceso por el cual *la información fluye entre 2 sistemas en 3 etapas*:

- **Extract:** se *consulta y extrae* del sistema origen.
- **Transform:** se *limpia, normaliza y procesa* para adaptarse al destino.
- **Load:** se *almacena ya transformada* en el destino.

ETL *no es propio de ninguna arquitectura particular*.

Sistema de facturación por sucursal → warehouse central con monto vendido por mes.

Extract: se obtienen las facturas del mes.

Transform: se calcula el total.

Load: se almacena el total en el warehouse.

ELT (Extract, Load, Transform)

Mismo objetivo que ETL pero *con orden distinto*:

- **Extract:** se *consulta y extrae* del sistema origen.
- **Load:** se *almacena ya transformada* en el destino.
- **Transform:** se *limpia, normaliza y procesa* para adaptarse al destino.

Sistema de facturación por sucursal → warehouse central con monto vendido por mes.

Extract: se obtienen las facturas del mes.

Load: se almacena el total crudo en el warehouse.

Transform: se calcula el total.